REDCap Avanzado — Material Sesión 1

Branching logic, campos calculados y Action Tags

Duración: 5 h · 09:00–14:00 · Requiere REDCap Básico o nivel equivalente

Agenda de la jornada

09:00–09:20	Bienvenida y comprobación de que cada participante tiene su instrumento (propio o el de prácticas "Hábitos de vida")
09:20–10:15	Branching logic: sintaxis y ejercicio guiado
10:15–11:15	Operadores, `and` / `or`, checkboxes y campos vacíos. Práctica sobre el instrumento propio
11:15–11:45	*Pausa*
11:45–12:45	Campos calculados: operaciones, funciones, IMC, edad, puntuaciones
12:45–13:20	Clasificaciones: `if()` anidado con códigos y con `@CALCTEXT`
13:20–13:50	Action Tags esenciales
13:50–14:00	Cierre y explicación del trabajo no presencial 1 (tabla de casos de prueba)

> **¿No traes instrumento propio?** Crea un proyecto vacío y, en `Dictionary → Upload`, sube el archivo `plantilla_habitos_vida_diccionario.csv` que se facilita con el curso. En un minuto tendrás un instrumento de práctica completo.

Bloque 1 — Branching logic

La branching logic se escribe en la propiedad `Branching Logic` (botón *Branching Logic* en el Designer) **del campo que se quiere mostrar u ocultar**, no en el campo del que depende. La condición describe cuándo se **muestra** el campo.

```
[nombre_variable] = 'código'
```

Convenciones de sintaxis

Elemento	Regla	Ejemplo
Variable	Siempre el `Variable Name` entre corchetes, nunca la etiqueta	`[edad]`, no `[Edad (años)]`
Código de opción	El **código**, no la etiqueta. Recomendado entre comillas simples	`[sexo] = '2'`
Número en comparaciones	Sin comillas, decimales con punto	`[imc] < 18.5` (nunca `18,5`)
Texto libre	Siempre entre comillas	`[municipio] = 'Peligros'`
`and` / `or`	REDCap acepta mayúsculas y minúsculas. Por convención usamos minúsculas	`[a] = '1' and [b] = '1'`

Espacios	Indiferentes alrededor de operadores y corchetes	`[edad]>=65` = `[edad] >= 65`

Ejercicio guiado: tu primera branching logic

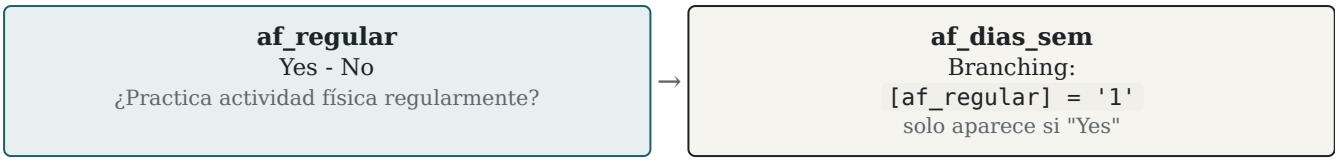


Figura 1. El campo dependiente solo se muestra cuando se cumple la condición escrita en su propiedad Branching Logic.

1. Campo base: `af_regular`, tipo `Yes - No` (REDCap codifica `1 = Yes`, `0 = No`).
2. Campo dependiente: `af_dias_sem`, Text Box, validación Integer, mínimo 1, máximo 7. Branching Logic: `[af_regular] = '1'`.
3. Prueba: responde "No" y la pregunta de días no aparece. Cambia a "Yes" y aparece.
4. Vuelve a "No" con los días ya rellenos: REDCap avisa de que el campo oculto tiene un valor y pregunta si se borra. **Responda que sí**, para no dejar datos huérfanos.

Operadores de comparación

Operador	Significado	Ejemplo
`=`	Igual	`[fuma_actual] = '1'`
`<>` (o `!=`)	Distinto	`[estado_civil] <> '1'`
`>` `<`	Mayor / menor que	`[edad] < 18`
`>=` `<=`	Mayor o igual / menor o igual	`[edad] >= 65`

Combinar condiciones

```
[sexo] = '2' and [embarazo] = '1'                          → las dos a la vez
[fuma_actual] = '1' or [convive_fumadores] = '1'            → al menos una
[edad] >= 65 and ([vive_solo] = '1' or [apoyo_familiar] = '0')
```

Los paréntesis agrupan lo que debe evaluarse primero. Cuando se mezclan `and` y `or`, póngalos siempre, aunque crea que no hacen falta: así la regla se entiende y no depende del orden de precedencia.

Checkboxes: la sintaxis especial

Un checkbox no tiene un único valor: cada opción es una variable 0/1. Por eso se pregunta por cada opción con el código entre paréntesis:

```
[barreras_af(2)] = '1'        → la opción 2 ("Coste") está MARCADA
[barreras_af(2)] = '0'        → la opción 2 NO está marcada
[barreras_af(1)] = '1' or [barreras_af(2)] = '1'   → marcó tiempo o coste
```

> **Error frecuente:** `[barreras_af] = '2'`. Con un checkbox **no funciona**, porque el campo no tiene un valor único. Es la causa más habitual de "mi regla no hace nada".

Campos vacíos

```
[peso_kg] <> ''        → mostrar solo si ya se ha respondido el peso
[peso_kg] = ''         → mostrar solo si está vacío (por ejemplo, para pedir el motivo)
```

Si la regla no funciona

- ☐ ¿He usado el `Variable Name` y no la etiqueta? ¿Está bien escrito?
- ☐ ¿El valor es el **código** de la opción (mirar en el Codebook)?
- ☐ ¿Es un checkbox? Entonces la sintaxis es `[campo(código)] = '1'`
- ☐ ¿Están cerrados todos los corchetes, paréntesis y comillas?
- ☐ ¿Decimales con punto?

Bloque 2 — Campos calculados

El tipo `Calculated Field` calcula su valor a partir de otros campos. Se actualiza en tiempo real mientras se rellena el formulario y no se puede editar a mano. La fórmula se escribe en `Calculation Equation`.

> **Un campo calculado solo puede devolver números.** Para obtener texto ("Normopeso", "Cumple") hay dos opciones: devolver un código numérico, o usar un campo de texto con `@CALCTEXT` (véase más abajo).

Operaciones y funciones útiles

Sintaxis	Qué hace
`+ - * / ^`	Suma, resta, multiplicación, división, potencia
`round(x, n)`, `rounddown(x, n)`, `roundup(x, n)`	Redondeo a *n* decimales
`sum(a, b, …)`, `mean(a, b, …)`	Suma o media **ignorando los vacíos**
`min()`, `max()`, `abs()`, `sqrt()`	Mínimo, máximo, valor absoluto, raíz cuadrada
`datediff([f1], [f2], "y")`	Diferencia entre dos fechas en años (`"y"`), meses (`"M"` , mayúscula) o días (`"d"`)
`if(condición, si_verdadero, si_falso)`	Condicional. La condición usa la misma sintaxis que la branching logic

> **Vacíos:** con `+`, si **un solo** sumando está vacío, el resultado queda vacío. Con `sum()`, los vacíos se ignoran. Elija según lo que quiera: en una escala, ¿debe puntuar un cuestionario incompleto?

Ejemplos aplicados

Campo	Calculation Equation
`af_min_sem` min/semana	`[af_dias_sem] * [af_min_sesion]` Prueba: 3 días × 45 min → 135
`imc` kg/m²	`round([peso_kg] / (([talla_cm] / 100) ^ 2), 1)` Prueba: 70 kg, 170 cm → 24.2. La talla en cm se divide entre 100.

`edad_calc` años cumplidos	`rounddown(datediff([fecha_nac], [fecha_inclusion], "y"), 0)` Use una fecha del propio registro, no `'today'`: el valor guardado no se actualiza solo con el paso del tiempo.
`dieta_med_punt` 0–7 puntos	`sum([ali_aceite], [ali_fruta], [ali_verdura], [ali_legumbre], [ali_pescado], [ali_carne_roja], [ali_dulces])` Cada ítem es Yes - No (1/0). Versión simplificada con fines didácticos: en investigación, use el cuestionario validado MEDAS de 14 ítems (PREDIMED).

Bloque 3 — Clasificar: códigos o `@CALCTEXT`

Opción A — Campo calculado que devuelve un código

```
if([imc] = '', '',
  if([imc] < 18.5, 1,
    if([imc] < 25, 2,
      if([imc] < 30, 3, 4))))
```

Se documenta en el `Field Note`: "1 = Bajo peso · 2 = Normopeso · 3 = Sobrepeso · 4 = Obesidad". Es la opción más cómoda para analizar después.

Opción B — Campo de texto con `@CALCTEXT`

Cree un campo **Text Box** (no Calculated) y escriba en su `Field Annotation`:

```
@CALCTEXT(
  if([imc] = '', '',
    if([imc] < 18.5, 'Bajo peso',
      if([imc] < 25, 'Normopeso',
        if([imc] < 30, 'Sobrepeso', 'Obesidad')))))
```

Es más legible para quien rellena el formulario, pero el resultado es texto. Para el análisis suele convenir tener también la opción A.

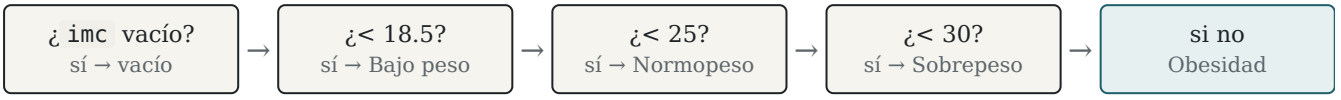


Figura 2. Cómo se evalúa un if() anidado: las condiciones se comprueban en orden y la primera que se cumple decide el resultado. Por eso basta con "< 25" en el segundo paso y no hace falta "≥ 18.5 y < 25".

Otra clasificación: recomendación de actividad física

```
Calculated:   if([af_min_sem] = '', '', if([af_min_sem] >= 150, 1, 0))
@CALCTEXT:    @CALCTEXT(if([af_min_sem] = '', '', if([af_min_sem] >= 150, 'Cumple', 'No
cumple')))
```

Umbral de referencia: OMS 2020, de 150 a 300 minutos semanales de actividad moderada en población adulta.

Dónde colocar los campos calculados

- Pueden ir en cualquier posición, incluso en otro instrumento. Por legibilidad, conviene ponerlos **después** de los campos que usan.
- Si el cálculo usa campos de **otro instrumento**, el valor se guarda cuando se guarda el formulario que contiene el cálculo.

- Si se cambia una fórmula cuando ya hay datos, los valores antiguos **no se recalculan solos**. Se corrigen con la regla de calidad "Incorrect values for calculated fields" (Sesión 2).

Bloque 4 — Action Tags esenciales

Las Action Tags se escriben en el `Field Annotation` del campo y modifican su comportamiento. Empiezan por `@` y van en mayúsculas. Se pueden combinar varias separadas por un espacio.

Action Tag	Qué hace	Uso típico
`@HIDDEN`	Oculta el campo en formularios y encuestas. Sigue existiendo y se puede usar en cálculos	Campos técnicos
`@HIDDEN-SURVEY`	Lo oculta solo en modo encuesta	Puntuaciones que el participante no debe ver
`@READONLY`	Visible, pero no editable	Valores fijados o precargados
`@DEFAULT='valor'`	Valor inicial si el campo está vacío	`@DEFAULT='1'` en el centro habitual
`@TODAY` / `@NOW`	Fecha (o fecha y hora) actual al abrir el formulario, si está vacío. `@TODAY` en campos de fecha; `@NOW` en fecha-hora	Fecha de cumplimentación, combinada con `@READONLY`
`@CALCTEXT(…)`	Cálculo que devuelve texto, en un campo Text Box	Clasificaciones legibles
`@NONEOFTHEABOVE='99'`	En un checkbox, convierte la opción 99 en excluyente: si se marca, no se pueden marcar las demás	"Ninguna de las anteriores"

Práctica de la sesión

Sobre tu propio instrumento (o el de prácticas):

☐ 2 reglas de branching logic, al menos una con `and`/`or` o con un checkbox

☐ 1 campo calculado con una función (`round`, `sum` o `datediff`)

☐ 1 clasificación con `if()`, como código numérico o con `@CALCTEXT`

☐ 1 Action Tag

☐ Todo probado con al menos un registro ficticio